

2. UART TX state machine (start + 8 data + stop)
3. Message scheduler (once per second / on press)
4. Optional hex encoder (turn nibbles into ASCII)

Step-by-step

1. **Decide clock and baud**
 - Common: 115200 baud.
 - Compute divider = $\text{CLK_HZ} / \text{BAUD}$ (integer).
2. **Implement UART TX**
 - Idle line high.
 - On `send`, shift out: start(0) + 8 data bits + stop(1).
3. **Create a simple “print engine”**
 - Store a short message in ROM (array of bytes), step through it.
4. **Hook it to your real pins**
 - Constrain `uart_tx` output pin.
 - Connect to a USB-UART dongle (3.3V logic!) or board’s onboard USB-serial.
5. **Build + program**
6. **Verify on PC**
 - Open a terminal (PuTTY/minicom/screen).
 - Set 115200, 8N1, no flow control.
7. **Debug**
 - Garbage characters: baud mismatch, wrong clock assumption, or inverted TX.
 - Nothing prints: wrong pin, missing ground between FPGA and dongle, or TX line not idle-high.

Upgrades

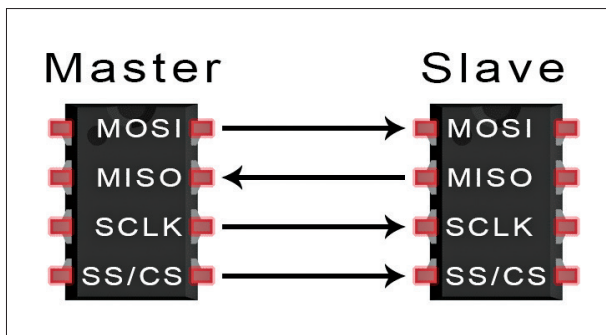
- Add button-triggered print: prints only on clean press.
- Add binary-to-hex conversion to display counters cleanly.

Portfolio packaging

- Screenshot of terminal output + a short note explaining UART frame basics.
- Project 4: SPI “sensor” reader (real peripheral communication)

What you’ll build

- SPI master that clocks out a read command and clocks in a byte/word



- If you have a sensor (or SPI flash), read a register ID
- If you don’t, use a mock SPI slave module on the FPGA to test end-to-end
 - SPI is a 4-wire workhorse: SCLK, MOSI, MISO, CS. SPI timing depends on clock polarity and phase (CPOL/CPHA), which define when data is sampled and shifted.

Blocks

1. SPI clock divider (creates SCLK from system clock)
2. SPI master FSM (IDLE → ASSERT_CS → SHIFT → DEASSERT_CS)
3. Shift registers (TX and RX)
4. Result display (LED pattern or UART print from Project 3)

Step-by-step

1. **Pick an SPI mode**
 - Start with Mode 0 (CPOL=0, CPHA=0) unless your device says otherwise. (ADI’s SPI intro explains CPOL/CPHA selection clearly.)
2. **Implement the master**
 - Pull CS low.
 - For each bit:
 - ◆ Drive MOSI.
 - ◆ Toggle SCLK edges.
 - ◆ Sample MISO on the correct edge for your CPHA.
3. **If you have hardware**
 - Wire CS/SCLK/MOSI/MISO to the peripheral.
 - Confirm voltage levels (most hobby SPI is 3.3V).
4. **If you don’t have hardware**
 - Instantiate a simple mock slave in RTL:
 - ◆ On CS low, shift out a fixed ID byte (e.g., 8’hA5) on MISO.
5. **Constrain pins**
 - If using off-board peripheral: map pins to headers.
 - If using mock: no extra pins needed; just show result via UART/LED.
6. **Build + program**
7. **Debug**
 - All zeros: MISO not connected, wrong sample edge, or CS not asserted.
 - Bit-shifted garbage: wrong CPHA (sampling on wrong edge).

Upgrades

- Add multi-byte reads (register address + data).
- Add a simple CRC/checksum in the received stream.

Portfolio packaging

- A timing diagram screenshot (simulation waveform) showing CS, SCLK, MOSI, MISO aligned to your chosen mode.